

INJIBARA UNIVERSITY

COLLEGE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF SOFTWARE ENGINERRING

LIBRARY MANAGEMENT SYSTEM

Group Members	ID
1. Etsegenet Bimrew	1700492
2. Fikrte Yenew	1700525
3. Baynesagn Andargachew	1700308
4. Muhedin Ararso	1700781
5. Haileeyesus Aschenk	1700608
6. Tsegaye Abebe	1700978

Advisor: Derso Mekuriaw

Injibara,Ethipoia
December 2018E.C

Table Content

Over view……3
1. Conceptual model……4
1.1 Entites……4
1.2 Relationships……5
1.3 Entity Relationship Diagram……6
2. Logical Model……7
2.1 Table and attributes……7
2.2 Logical model Relation ship……9
3. Physical Model……10
3.1 Table definition……11
3.2 Data Insertion……12
3.3 Data retrival……14
Conclusion……15
Reference……16

Overview

The library Management system is designed to manage and organize library activities such as book registration,member management,borrowing,returning and fine calculation.the system handles information about books, authors,categories,members,loans and fines it helps the library store information accurately and reduce manual work.

In this system each book belongs to a specific category and may be written by one or more author. Members can borrow books and every borrowing activities is recorded as loan. The loan entity stores important details such as loan date,due date and return date. If a book is returned late the system generates a fine linked to that loan.

The database design follows three stages: conceptual,logical and physical modeling.this ensures that the system is well-structured avoids data duplication and maintains data integrity. Overall the library management system improves efficiency,reduces manual work and provides a reliable way to manage library resource.

1. Conceptual Model

The Library Management System (LMS) is designed to manage the core operations of a library, including book management, member management, loans, reservations, and fines. The system's conceptual model identifies the main entities, their attributes, and the relationships between them without considering implementation details such as tables or data types.

1.1 Entities:

- ***Member –*** Represents a library user.who can borrow or reserve book.
  Attributes: Member ID, First Name, Last Name, and Phone Number. Members can borrow and reserve books.

- ***Book –*** Represents library books.
  Attributes: Book ID, Title,Book Status, and Category ID.

- ***Author –*** Represents the writer of books.
  Attributes: Author ID, First Name, and Last Name.

- ***Category –*** Represents the classification of books (e.g., Science, Literature).
  Attributes: Category ID and Name.

- ***Loan –*** Represents the act of borrowing a book by a member.
  Attributes : Loan ID, Loan Date, Return Date,Due Date,Book ID,Member ID.

- ***Reservation –*** Represents a request by a member to reserve a book.
  Attributes: Reservation ID , Reservation Date,Book ID,Member ID.

- ***Fine –*** Represents penalties for late returns.
  Attributes : Fine ID, Amount, Fine Date, Payment Date,Loan ID.

1.2 Relationships:

Member–Loan: Members borrow books through loans. A loan is linked to exactly one member and one book.

Book-Loan: Each book can be borrowed multiple times over time. Each loan is for exactly one book.

Loan -Fine:A loan may generate a fine if the book is returned late. Each fine corresponds to exactly one loan

Member–Reservation: Members reserve books. A member can make multiple reservations, and a reservation applies to one member.

Book -Reservation: A book can have multiple reservation.each reservation is exactly to one book

Book–Author: Books are written by authors. A book can have multiple authors, and an author can write multiple books

Book–Category: Each book belongs to a single category, while a category can have multiple books.

This conceptual model clearly defines the scope of the system, highlighting entities, their key attributes, and how they interact in a real-world library context. It serves as the foundation for designing the logical and physical models.

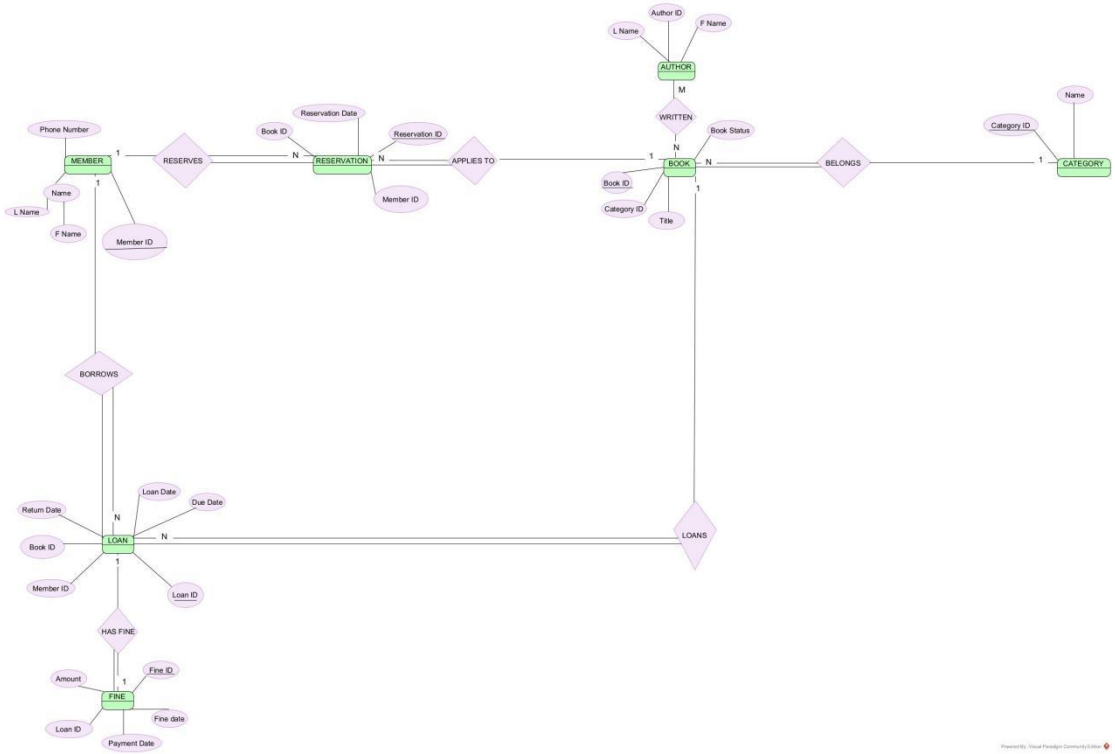


Diagram 1: Entity Relationship Model

2. Logical Model

Logical data model is representation of data elements and their relationships. It helps structure and organize data independently of specific database technologies.
Key components of logical data model include entities,attributes and relationships. Logical data model ensure data integrity and consistency across the system.
The logical model for our library management system is normalized up to third normal form (3NF) ensuring minimal redundancy,strong data integrity and proper dependency management.

2.1 Tables and Attributes

✧ **Member**

- **PK**: member_id (NOT NULL, UNIQUE)
- first_name (NOT NULL)
- last_name (NOT NULL)
- phone_number

✧ **Book**

- **PK**: book_id (NOT NULL, UNIQUE)
- title (NOT NULL)
- book_status (NOT NULL)
- category_id (FK to Category.category_id, NOT NULL)

✧ **Author**

- **PK**: author_id (NOT NULL, UNIQUE)
- first_name (NOT NULL)

- last_name (NOT NULL)

✧ **Category**

- **PK:** category_id (NOT NULL, UNIQUE)
- name (NOT NULL, UNIQUE)

✧ **Loan**

- **PK: l**oan_id (NOT NULL, UNIQUE)
- member_id (FK to Member.member_id, NOT NULL)
- book_id (FK to Book.book_id, NOT NULL)
- loan_date (NOT NULL)
- due_date (NOT NULL)
- return_date (NULL allowed)

✧ **Reservation**

- **PK:** reservation_id (NOT NULL, UNIQUE)
- member_id (FK to Member.member_id, NOT NULL)
- book_id (FK to Book.book_id, NOT NULL)
- reservation_date (NOT NULL)

✧ **Fine**

- **PK:** fine_id (NOT NULL, UNIQUE)
- loan_id (FK to Loan.loan_id, NOT NULL)

- amount (DEFAULT 0, NOT NULL)
- fine_date (NOT NULL)
- payment_date

✧ **Book_Author (Junction Table for Many-to-Many between Book and Author)**

- **PK:** (book_id, author_id) (NOT NULL)
- **FK**: book_id → Book.book_id
- **FK**: author_id → Author.author_id

2.2 Logical Model Relationships

1. Member and Loan

Each loan belongs to exactly one member.
A member can have one or many loans over time.
Cardinality: 1 :N

2. Book and Loan

Each loan is for exactly one book.
A book can be loaned multiple times, but only once per loan record.
Cardinality: 1 :N

3. Loan and Fine

A loan may generate one or one fine if the book is returned late.
Each fine is linked to exactly one loan.
Cardinality: 1 :1

4. Member and Reservation

Each reservation is made by one member.
A member can make one or many reservations.
Cardinality: 1 :N

5. Book and Reservation

Each reservation is for exactly one book.
A book can have one or many reservations.
Cardinality: 1 :N

6. Book and Category

Each book belongs to exactly one category.
Each category can include many books.
Cardinality: N:1

7. ***Book and Author***

A book can have multiple author.
An author can write multiple books.
Cardinality:N:M

3. Physical Model

The Physical Model represents the database design ready for implementation in a relational database system. It defines tables, columns, data types, primary keys, foreign keys, and constraints to ensure data integrity and efficient storage. This model clearly establishes relationships between entities and enforces business rules, supporting all library operations such as book loans, reservations, and fines. By translating the logical design into a concrete structure, it provides a blueprint for building the database that ensures accuracy, consistency, and reliability.

3.1 Table Definition

- ***Member Table***

```
CREATE TABLE Member (
   member_id  INT PRIMARY KEY UNIQUE,
   first_name VARCHAR(50) NOT NULL,
   last_name VARCHAR(50) NOT NULL,
   phone_number INT );
```

- ***Book Table***

```
CREATE TABLE Book (
   book_id INT PRIMARY KEY UNIQUE,
   title VARCHAR(100) NOT NULL,
   book_status VARCHAR(20) NOT NULL,
   category_id INT NOT NULL,
   FOREIGN KEY (category_id) REFERENCES Category(category_id));
```

- ***Author Table***

```
CREATE TABLE Author (
   author_id INT PRIMARY KEY UNIQUE,
   first_name VARCHAR(50) NOT NULL,
   last_name VARCHAR(50) NOT NULL);
```

- ***Category Table***

```
CREATE TABLE Category (
   category_id INT PRIMARY KEY UNIQUE,
   name VARCHAR(50) NOT NULL UNIQUE);
```

- ***Loan Table***

```
CREATE TABLE Loan (
   loan_id INT PRIMARY KEY UNIQUE,
   member_id INT NOT NULL,
   book_id INT NOT NULL,
   loan_date DATE NOT NULL,
   due_date DATE NOT NULL,
   return_date DATE NULL,
```

```
  FOREIGN KEY (member_id) REFERENCES Member(member_id),
  FOREIGN KEY (book_id) REFERENCES Book(book_id));
```

- ***Fine Table***

```
CREATE TABLE Fine (
  fine_id INT PRIMARY KEY UNIQUE,
  loan_id INT NOT NULL,
  amount DECIMAL(10,2) NOT NULL DEFAULT 0,
  fine_date DATE NOT NULL,
  payment_date DATE NULL,
  FOREIGN KEY (loan_id) REFERENCES Loan(loan_id));
```

- ***Reservation Table***

```
CREATE TABLE Reservation (
  reservation_id INT PRIMARY KEY UNIQUE,
  member_id INT NOT NULL,
  book_id INT NOT NULL,
  reservation_date DATE NOT NULL,
  FOREIGN KEY (member_id) REFERENCES Member(member_id),
  FOREIGN KEY (book_id) REFERENCES Book(book_id));
```

- ***Book_Author (Junction Table)***

```
CREATE TABLE Book_Author (
  book_id INT NOT NULL,
  author_id INT NOT NULL,
  PRIMARY KEY (book_id, author_id),
  FOREIGN KEY (book_id) REFERENCES Book(book_id),
  FOREIGN KEY (author_id) REFERENCES Author(author_id));
```

3.2 Data Insertion

- ***Insert data into Category table***

```
  INSERT INTO Category (category_id, name)
  VALUES
  (1, 'Computer Sci'),
  (2, 'Programming'),
  (3, 'Web Dev');
```

✧ ***Insert data into Member table***

```
INSERT INTO Member (member_id, first_name, last_name,
phone_number) VALUES
(1, 'Sofna', 'Dagm', 0912345678),
(2, 'Sara', 'Eyosiyas’, 0923456789),
(3, 'Hawa', 'Jemal', 0934567890);
```

✧ ***Insert data into Book table***

```
INSERT INTO Book (book_id, title, book_status, category_id)
 VALUES
(1, 'Introduction to AI', 'Available', 1),
(2, 'Data Structures in C', 'Loaned', 2),
(3, 'Modern Web Development', 'Available', 3);
```

✧ ***Insert data into Author table***

```
INSERT INTO Author (author_id, first_name, last_name)
 VALUES
(1, 'Alan', 'Turing'),
(2, 'Robert', 'Martin'),
(3, 'John', 'Resig');
```

✧ ***Insert data into Loan table***

```
INSERT INTO Loan (loan_id, member_id, book_id, loan_date, due_date,
return_date)
VALUES
(1, 1, 2, '2025-12-10', '2025-12-20', NULL),
(2, 2, 1, '2025-12-12', '2025-12-22', '2025-12-21'),
(3, 3, 3, '2025-12-15', '2025-12-25', NULL);
```

✧ ***Insert data into Fine table***

```
INSERT INTO Fine (fine_id, loan_id, amount, fine_date, payment_date)
VALUES
(1, 1, 5.00, '2025-12-21', NULL),
(2, 2, 0.00, '2025-12-22', '2025-12-22');
```

✧ ***Insert data into Reservation table***

```
INSERT INTO Reservation (reservation_id, member_id, book_id,
reservation_date)
 VALUES
(1, 1, 1, '2025-12-20'),
(2, 2, 3, '2025-12-21');
```

✧ ***Insert data into Book_Author junction table***

```
INSERT INTO Book_Author (book_id, author_id)
VALUES
(1, 1),
(2, 2),
(3, 3),
(3, 2);
```

3.3 Data Retrival

```
SELECT *FROM Member;

SELECT *FROM Category;

SELECT *FROM Author;

SELECT *FROM Fine;

SELECT *FROM Loan;

SELECT *FROM Book_Author;

SELECT *FROM Reservation;

SELECT *FROM Book;
```

Conclusion

In this project, a Library Management System was designed using conceptual, logical, and physical data models. The conceptual model helped to understand the real-world entities and their relationships, while the logical model transformed these ideas into structured relations with keys and constraints. Finally, the physical model defined the actual database implementation using tables, data types, and integrity constraints.

The system efficiently manages core library operations such as book management, member registration, loans, reservations, and fines, while ensuring data integrity and consistency through proper use of primary keys, foreign keys, and normalization up to the third normal form (3NF). This design provides a clear, scalable, and reliable foundation for implementing a real-world library database system.

Reference

ER/Studio
https://erstudio.com
Redgate
https://www.red-gate.com
GoodDate
https://www.gooddata.com
Tibco
https://www.tibico.com
IBM
https://www.ibm.com
datamation.com
https://www.datamation.com
Amazon Web Services(AWS)
https://aws.amazon.com
@NextLVLprograming
https://Youtube.com